Kegiatan Belajar 4F.3: Migrasi Frontend ke API

Hari 14 · 8 JP · UK 1.1, 1.2, dan 1.5

Tujuan KB 4F.3. Setelah mengikuti KB ini, Anda mampu memindahkan sumber data `app.js` dari berkas JSON ke API PHP tanpa mengubah tampilan, serta memahami konsep **kontrak data** antara frontend dan backend.

Kegiatan	**JP**
Konsep kontrak data (API contract) antara frontend dan backend	1
Praktik: mengubah satu baris di `app.js`	1
Pengujian menyeluruh fitur publik Kopi Nusantara	3
Menangani kegagalan server dengan baik	2
Cek kemampuan	1
Jumlah	**8**

Materi 1: Kontrak Data (*API Contract*)

Bentuk JSON yang dikembalikan `api/produk.php` **sama persis** dengan bentuk `data/produk.json` yang lama. Ketika frontend (JavaScript) dan backend (PHP) "sepakat" soal bentuk data yang dipertukarkan, perubahan di satu sisi **tidak perlu** memaksa perubahan besar di sisi lain — inilah yang disebut **kontrak data**. Karena kontraknya dijaga tetap sama, migrasi seharusnya hanya butuh **mengubah alamat yang dipanggil `fetch()`**.

Praktik 1: Satu Baris Perubahan

Buka `js/app.js`, cari fungsi `muatData()`:

```
async function muatData() {
  const respons = await fetch("data/produk.json");
  // ...
}
```

Ubah **hanya** alamatnya:

```
async function muatData() {
  const respons = await fetch("api/produk.php");
  // ...
}
```

Simpan, segarkan browser. Karena `.htaccess` di KB 4F.2 sudah memblokir akses langsung ke `data/produk.json`, baris lama **pasti** gagal jika belum diubah — perubahan ini bukan sekadar "praktik baik", tapi **sekarang wajib** agar halaman bisa berjalan sama sekali.

Praktik 2: Pengujian Menyeluruh

- ☐ Menu navigasi tampil dan tautannya berfungsi.
- ☐ Seluruh kartu produk tampil dengan data yang benar.
- ☐ Filter kategori tetap bekerja.
- ☐ Formulir kontak tetap memvalidasi isian kosong.
- ☐ Tahun di footer tetap muncul otomatis (bukti Temuan 1 di KB 4F.1 sudah benar-benar beres).

- ☐ Tidak ada galat merah di Console.
- ☐ Tab **Network** di DevTools menunjukkan permintaan ke `api/produk.php` (bukan lagi ke `data/produk.json`), dengan status `200`.
- ☐ Mencoba membuka `data/produk.json` langsung tetap menghasilkan 403 Forbidden (proteksi tidak sengaja ikut terlepas).

Materi 2: Menangani Kegagalan dengan Baik

Pola loading/sukses/error dari Modul 3 tetap relevan di sini — hanya penyebab galatnya berbeda dari versi basis data: **berkas JSON yang rusak formatnya**, bukan koneksi basis data yang terputus.

Coba secara sengaja (lalu kembalikan setelah diamati): buka `data/produk.json` di VS Code, hapus satu tanda kurung kurawal penutup `}`, simpan. Segarkan Kopi Nusantara di browser. Amati: fungsi `bacaJson()` di `json_store.php` dirancang mengembalikan array kosong (`[]`) jika `json_decode` gagal (lihat `?? []` di KB 4F.2) — artinya `api/produk.php` tetap mengembalikan JSON yang **valid** (`[]`), tidak pernah mengembalikan galat PHP mentah ke browser. Periksa: apakah halaman menampilkan "tidak ada produk" dengan wajar, atau malah rusak total? Perbaiki `produk.json` kembali setelah selesai menguji.

Latihan 3

1. Lakukan eksperimen "rusak sengaja" pada Materi 2 di atas, catat apa yang terjadi di halaman dan di Console.
2. Pada proyek **tema sendiri** Anda, ulangi seluruh langkah KB 4F.2 dan 4F.3: proteksi `data/`, buat endpoint, migrasikan `app.js`.
3. *Diskusi:* dengan `bacaJson()` yang "diam-diam" mengembalikan array kosong saat JSON rusak, galat semacam ini **tidak pernah terlihat** oleh pengguna maupun di Console sebagai error tegas. Apa untung-ruginya desain seperti ini dibanding membuat `api/produk.php` sengaja mengembalikan kode galat 500 saat JSON rusak?

Rangkuman KB 4F.3

- Kontrak data yang dijaga konsisten membuat migrasi sumber data hanya butuh perubahan minimal di frontend.
- Setelah `.htaccess` aktif, migrasi ke API **bukan lagi pilihan** — ini satu-satunya cara halaman tetap berfungsi.
- `bacaJson()` yang mengembalikan array kosong saat JSON rusak mencegah kegagalan total, tapi juga berarti kesalahan format JSON bisa "tersembunyi" dari pengguna — kompromi yang perlu dipahami, bukan dianggap sempurna.

Cek Kemampuan KB 4F.3

- ☐ Kopi Nusantara berjalan sepenuhnya dari `api/produk.php`, seluruh daftar periksa Praktik 2 terpenuhi.
- ☐ Proyek tema sendiri saya juga sudah bermigrasi ke API dan folder `data/`-nya terproteksi.
- ☐ Saya sudah menguji skenario JSON rusak, dan memahami perilaku `bacaJson()` saat itu terjadi.
- ☐ Saya dapat menjelaskan istilah "kontrak data" dengan kata sendiri.

Kesalahan umum: lupa mengembalikan `produk.json` ke kondisi semula setelah eksperimen "rusak sengaja"; alamat `fetch()` salah ketik; menguji hanya tampilan tanpa memeriksa tab Network untuk memastikan sumber data sungguhan sudah berubah.